

EvoCrypt: An Upgradeable Cryptographic Coprocessor Framework for the Ethereum Virtual Machine

Qi Liu¹, Qianhong Wu¹, Bo Qin², and Hongzi He¹

¹ School of Cyber Science and Technology, Beihang University, Beijing 100191, China

² School of Information, Renmin University of China, Beijing 100872, China

Abstract. Smart contracts frequently rely on cryptographic algorithms, yet implementing complex cryptographic computation in Solidity expands it into large sequences of EVM instructions, resulting in substantial execution overhead and gas consumption. Precompiled Contracts reduce these costs by exposing client-side routines at fixed addresses, but they provide no runtime mechanism for evolving an algorithm independently of client releases and coordinated network upgrades. We present EvoCrypt, an upgradeable cryptographic coprocessor framework that exposes a stable contract-facing interface while executing replaceable WebAssembly (WASM) modules in a controlled client-side runtime. EvoCrypt distributes version metadata on-chain and deterministically activates an installed module at a designated block height, allowing participating nodes to switch algorithm versions consistently. Each invocation is charged according to a deterministic algorithm-level gas schedule associated with the activated version, thereby avoiding the cost of expanding the algorithm into EVM instructions while retaining explicit resource accounting. We implement EvoCrypt in Geth and evaluate it on a 20-node private Clique network configured with zero-period block production to isolate system-level upgrade overhead from the block interval. Measured *network-wide upgrade duration* is the Geth-side time to finish upgrade handling on all evaluated nodes—from RPC submission through receipt observation, event handling, and node-local WASM preparation—and excludes network propagation and consensus delay; under this definition the prototype completes upgrades within 0.54–2.34 ms across the eight evaluated algorithms. EvoCrypt achieves execution latency comparable to Solidity implementations on lightweight algorithms, with up to 48% lower latency on computation-heavy algorithms.

Keywords: blockchain · smart contracts · upgrade · WebAssembly

1 Introduction

As blockchain systems become programmable platforms, smart contracts increasingly depend on cryptographic algorithms for authentication, integrity, signatures, and privacy [12, 24, 20, 6]. The long-term security of these applications

also requires the ability to adopt new algorithms, including post-quantum alternatives, without disrupting contract-facing functionality [18, 8, 13]. Blockchain clients therefore need an extensible way to provide evolving cryptographic capabilities to smart contracts.

Ethereum provides two main mechanisms for executing cryptographic operations: Solidity contract implementations and Precompiled Contracts. Solidity implementations are flexible and deployable at the contract layer, but complex cryptographic computation expands into costly EVM execution; Precompiled Contracts execute selected routines efficiently in client-side code, but their supported algorithms are fixed to the client and normally require coordinated client updates to change [20, 6, 7].

Together, these mechanisms expose a fundamental trade-off between execution efficiency and algorithm upgradability. Solidity keeps algorithms deployable without client changes but can impose substantial execution and gas overhead, whereas Precompiled Contracts provide efficient execution only for routines embedded in the client and are therefore difficult to evolve independently. The central challenge is to replace contract-facing cryptographic implementations without sacrificing efficient execution, deterministic behavior across nodes, or compatibility with the existing EVM interface.

EvoCrypt addresses this trade-off by separating the stable contract-facing interface from replaceable WASM implementations and by separating module preparation from module activation. The interface preserves how smart contracts invoke cryptographic operations, while portable WASM modules allow implementations to be replaced without embedding each algorithm in contract bytecode or rebuilding the client for every change. Asynchronous preparation moves verification, compilation, and instantiation away from the transaction execution path. Activation-Block-based selection then ensures that nodes switch versions according to a common chain-derived rule rather than local readiness time. The design combines algorithm-level accounting with deterministic, incrementally deployable upgrades.

The main contributions of this article are summarized as follows.

- First, we design an upgradeable cryptographic coprocessor that separates the stable contract-facing interface from replaceable WASM-based algorithm implementations, allowing complex cryptographic operations to execute outside the EVM while preserving contract compatibility.
- Second, we design an activation-block-based upgrade protocol in which upgrade proposals are published on-chain, ensuring that clients derive the same proposal from a common authoritative record. By distributing and verifying a uniform WASM bytecode artifact, the protocol further ensures consistent module execution across clients when the upgrade becomes active.
- Finally, we implement EvoCrypt in Geth and evaluate it on a 20-node private blockchain network. The results show reduced gas execution cost for the evaluated cryptographic operations and demonstrate support for algorithms that are difficult to implement efficiently in Solidity.

2 Related Work

Research on cryptographic algorithm upgrades in blockchain systems has not yet resolved the combined problem of efficient invocation and practical implementation. Related approaches can be organized by the layer at which they support upgrades: protocol and client, contract, or dynamic runtime.

Existing blockchain upgrade schemes primarily replace code at the protocol or client level, where a coordinated version transition preserves a common execution rule across nodes. Cardano’s hard-fork combinator moves the chain between protocol eras at designated upgrade points, while work on updatable blockchains studies synchronization between old and new chains [10, 4, 5]. Soft-fork approaches instead place selected policy changes within rules that can be adopted without replacing the entire protocol [23]. Phoenix extends this direction by embedding upgrade code in blockchain transactions and compiling it at runtime. Polkadot stores WASM-based runtime logic on chain and activates governance-approved replacements during block synchronization [19, 15]. These approaches support chain-wide or client-wide evolution, but they require protocol coordination and place the replacement boundary across a substantial portion of system execution.

Contract-level schemes update application logic without replacing the entire blockchain client, using deployable contracts, proxy indirection, or bytecode rewriting. EVMPatch patches deployed Ethereum smart contracts through bytecode rewriting and proxy-based deployment, while Solidity implementations offer application-level flexibility within the EVM [17, 6]. LEAGAN extends contract-level upgradeability through data separation, incremental hashing, revision control, and status-contract-based version management [11]. Precompiled Contracts provide a complementary client-side path for selected cryptographic routines through reserved EVM-facing addresses [7, 16, 3]. Contract-level implementations remain constrained by EVM execution cost and state or interface compatibility, whereas adding or replacing a Precompiled Contract normally requires changes to client and protocol rules.

Dynamic software-update schemes move replacement below the application layer by loading new components into a running host, with emphasis on availability, version compatibility, or isolation. KylinX supports runtime library replacement through dynamic mapping, Mvedsua combines dynamic updates with multi-version execution, PYLIVE changes Python code online, and distributed-system research addresses modular updates and version consistency [22, 14, 9, 1, 2]. WASM provides a portable artifact format and explicit host boundary for this style of replacement. In blockchain systems, existing WASM-based upgrades generally target broad runtime logic rather than a narrow EVM-facing cryptographic interface [21, 15].

3 EvoCrypt Design

EvoCrypt is a cryptographic coprocessor framework for dynamically upgrading contract-facing cryptographic algorithms. This section presents the system

model, the upgrade protocol, and the invocation protocol that together define the design. We focus on the coordination, version-selection, and contract-facing execution rules of EvoCrypt, while the prototype implementation and experimental configuration are described in Section 5.

3.1 System Model

EvoCrypt separates governance, chain state, contract execution, and module execution into four cooperating components: the Community, the Blockchain, the EVM, and the Cryptographic Coprocessor. The Community includes developers and an authorizing committee that proposes and approves algorithm upgrades. The Blockchain maintains the canonical chain and shared world state, including smart contracts and the Management Contract. The EVM processes smart-contract transactions, while the node-local Cryptographic Coprocessor prepares, activates, and executes replaceable cryptographic modules.

Two chain-derived paths connect these components. An approved proposer records an upgrade through the Management Contract, whereas a contract initiates an invocation through the EVM of a Blockchain node. The Management Contract stores algorithm metadata and exposes the contract-facing upgrade and invocation interfaces. The coprocessor uses the resulting chain state to prepare and execute the corresponding module. Figure 1 summarizes these objects and their relationships.

3.2 Upgrade Protocol

On-chain publication. On-chain publication provides the authoritative record that coordinates node-local preparation and height-based activation. After the committee approves a proposal, a committee-authorized proposer submits it through the designated upgrade interface. The Management Contract admits the proposal only after the committee has reached consensus on its contents, stores the version metadata, and emits an upgrade event.

The EVM records metadata but does not compile or instantiate the WASM module. Node-dependent preparation therefore remains outside the transaction that changes the chain state. Every node reconstructs the same version from the admitted record, which identifies the algorithm, version, WASM artifact, module hash, interface descriptors, resource parameters, and Activation Block. Only $\langle a, v, H_{a,v} \rangle$ is emitted as a notification index. Let $\mathcal{V}_a$ denote the versions already registered for algorithm a ; the authoritative record prevents nodes from preparing different artifacts or execution rules for the same upgrade.

Definition 1 (Version metadata). *Let a be an algorithm name and v a version number. An admitted upgrade is the metadata record*

$$\mu_{a,v} = (a, v, C_{a,v}, h_{a,v}, g_{a,v}^{(0)}, \lambda_{a,v}, \tau_{a,v}^{\text{in}}, \tau_{a,v}^{\text{out}}, H_{a,v}), \quad (1)$$

where $C_{a,v}$ is the compiled WASM bytecode, $h_{a,v} = \text{Keccak256}(C_{a,v})$ is the module hash, $g_{a,v}^{(0)}$ is the initial gas fee, $\lambda_{a,v}$ is the gas slope parameter, $\tau_{a,v}^{\text{in}}$ and $\tau_{a,v}^{\text{out}}$

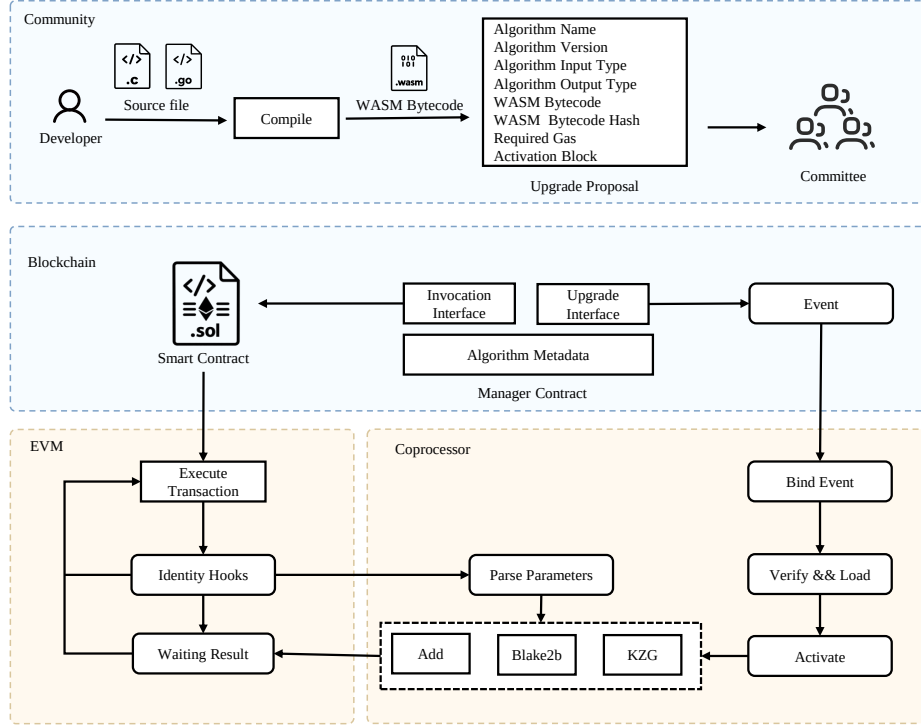


Fig. 1. System model for proposed EvoCrypt framework.

are the ABI input and output type descriptors, and $H_{a,v}$ is the Activation Block height. The Management Contract stores an encoding of these fields. Each node recomputes $h_{a,v}$ from the recovered bytecode during preparation and binds that hash to the prepared version.

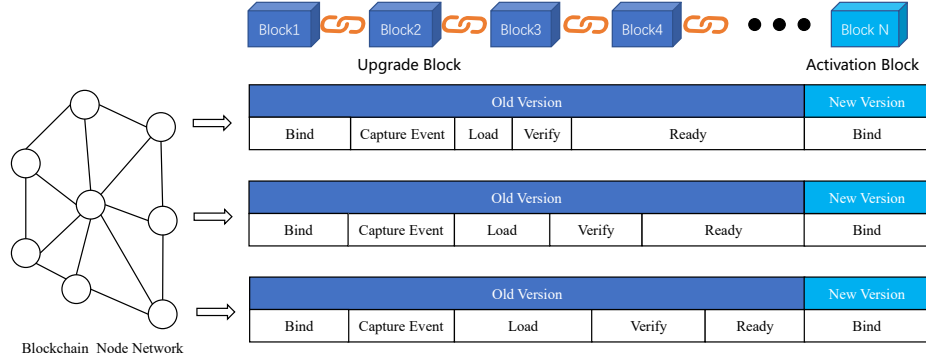
Node-local preparation. Node-local preparation converts the admitted meta-data record into executable state without changing the active version. After observing the upgrade event, each client recovers $\mu_{a,v}$, checks that the event index agrees with the stored record, decodes the artifact, and recomputes its hash. The Upgrade Handler then compiles and instantiates the module and validates its required entry point. Once these checks succeed, the node records (a, v) as prepared. Algorithm 1 gives the complete publication and preparation procedure.

Preparation is deliberately separated from the chain-level upgrade record. Node-local work can proceed asynchronously, failed attempts can retry from the authoritative chain record, and an executed upgrade transaction is not rolled back. The separation allows preparation to complete without changing the version selected by the chain.

Height-based activation. Height-based activation makes the selected version a function of shared chain height rather than node-local preparation time. The Upgrade Block includes the admitted upgrade transaction and emits $\langle a, v, H_{a,v} \rangle$.

Algorithm 1 Activation-Block-based WASM upgrade protocol.**Require:** Metadata $\mu_{a,v}$ as in (1); node set $\mathcal{N}$ **Ensure:** Prepared set updated on every $n_i \in \mathcal{N}$

- 1: committee-authorized proposer submits $\mu_{a,v}$ through the designated upgrade interface
- 2: Management Contract stores $\mu_{a,v}$; emits event $\langle a, v, H_{a,v} \rangle$
- 3: **for all** $n_i \in \mathcal{N}$ **do**
- 4: observe $\langle a, v, H_{a,v} \rangle$; fetch $\mu'_{a,v}$ from Management Contract
- 5: **if** event fields disagree with $\mu'_{a,v}$ **then**
- 6: reject (a, v)
- 7: **else**
- 8: decode $C_{a,v}$; set $h_{a,v} \leftarrow \text{Keccak256}(C_{a,v})$
- 9: **repeat**
- 10: compile and instantiate; validate the required module entry point
- 11: **until** preparation succeeds
- 12: persist $h_{a,v}$ and $\mu_{a,v}$; mark (a, v) prepared
- 13: **end if**
- 14: **end for**

**Fig. 2.** Node-local preparation and deterministic network-wide activation of an EvoCrypt upgrade.

After that block, each node independently completes the off-chain Load–Verify–Ready sequence, while invocations before $H_{a,v}$ continue to use the previously selected version. At the Activation Block, nodes with the same chain view select version v according to the rule below. Compilation-time differences therefore cannot change the version selected at a given execution height.

When a selected version is not yet prepared, the node reloads its authoritative metadata and continues preparation rather than selecting an obsolete version. The policy preserves height-based selection while allowing node-local readiness to complete asynchronously. Figure 2 shows the separation between publication, preparation, and activation.

Definition 2 (Active version). Let $\mathcal{V}_a$ be the registered version set of algorithm a , and let b be the execution block height. The version used at height b is

$$v^*(a, b) = \arg \max_{v \in \mathcal{V}_a} \{(H_{a,v}, v) \mid H_{a,v} \leq b\}. \quad (2)$$

The $\arg \max$ is lexicographic on the pair $(H_{a,v}, v)$: among versions whose activation height does not exceed b , the rule prefers the latest activation height and, for a tied height, the larger version number. If the feasible set is empty, no version is active and the invocation fails with a missing-version error.

3.3 Invocation Protocol

A unified Management Contract entry point preserves a stable contract-facing invocation boundary. A caller serializes the algorithm name a and its parameters into an encoded request x and submits the request through the designated interface. Different algorithms can therefore be invoked without changing the destination address, while upgrade and invocation events remain associated with one on-chain management endpoint. The contract depends on the ABI descriptors of the selected version rather than on the module location, implementation language, or node-local loading state.

Version resolution precedes execution. If the selected version is not yet prepared, the coprocessor prepares it rather than redirecting the request to an obsolete implementation.

EVM-side Hooks determine whether a request enters the Cryptographic Coprocessor. Requests that do not target the designated invocation interface remain on the ordinary EVM execution path. For a routed request, the coprocessor parses (a, x) , resolves $v^*(a, b)$ by (2), and prepares the selected version if necessary. The selected module receives the encoded parameters and returns an encoded result; the contract-facing call shape remains compatible when $\tau_{a,v}^{\text{in}}$ and $\tau_{a,v}^{\text{out}}$ are compatible.

Routing is followed by version resolution, optional preparation, deterministic charging, and module execution. Algorithm 2 specifies this post-routing procedure and its protocol-error conditions.

Definition 3 (Algorithm-level gas schedule). Each algorithm version uses an independent deterministic pricing rule so that its fee can reflect the resource class of the algorithm while remaining identical across clients. A call with inner input x against version v of algorithm a is charged

$$G(a, v, x) = g_{a,v}^{(0)} + \lambda_{a,v} \cdot |x|, \quad (3)$$

where $|x|$ is the byte length of the call-stage inner input, $g_{a,v}^{(0)}$ is the initial gas fee, and $\lambda_{a,v}$ is the gas slope parameter registered in $\mu_{a,v}$. The charged amount does not depend on node-local CPU time or on the number of WASM instructions executed inside the module.

Algorithm 2 Contract-to-coprocessor invocation protocol.**Require:** Recognized invocation request d , block height b , gas budget g **Ensure:** ABI-encoded result y or explicit error

```

1:  $(a, x) \leftarrow \text{Parse}(d)$ 
2: if the request is invalid then
3:   return ABI error
4: end if
5:  $v \leftarrow v^*(a, b)$  by (2)
6: if no applicable version exists then
7:   return missing-version error
8: end if
9: ensure  $(a, v)$  is prepared under the mandatory-preparation policy
10:  $G_{\text{req}} \leftarrow G(a, v, x)$  by (3)
11: if  $g < G_{\text{req}}$  then
12:   return out-of-gas error
13: end if
14: charge  $G_{\text{req}}$ ;  $y \leftarrow \text{Execute}(a, v, x)$ 
15: return  $y$ , or runtime error on failure

```

For each algorithm version, the algorithm-level schedule accounts for input-sensitive resource demand without relying on node-local measurements. Its coefficients assign a fixed cost, a length-dependent cost, or both to each algorithm version. Consequently, the schedule remains independent of local CPU time, runtime fluctuations, and the number of WASM instructions observed by an individual client.

With this fixed-plus-length form, every node that resolves the same $v^*(a, b)$ and receives the same input charges an identical fee. The schedule follows the deterministic pricing intuition of Precompiled Contracts while retaining an input-sensitive component. Coefficients used in Section 5 demonstrate the pricing mechanism rather than a calibrated economic optimum.

4 Security Analysis

This chapter establishes the security boundaries of EvoCrypt under explicit trust assumptions. It explains how canonical metadata and deterministic version selection support upgrade and invocation consistency, and how the WASM runtime limits host-resource exposure while preserving online governance.

4.1 Security Model and Assumptions

The security model covers the blockchain, the Management Contract state, and the node-local coprocessor. For an algorithm a and version v , let $\mu_{a,v}$ denote the authoritative metadata record, let $C_{a,v}$ denote its encoded WASM bytecode, let $h_{a,v}$ denote the recorded module hash, let $H_{a,v}$ denote the Activation Block, and let $v^*(a, b)$ be the version selected at execution height b according to (2).

Security claims concern honest nodes that execute the same canonical chain and therefore observe the same committed Management Contract state.

Each admitted proposal is assumed to be registered in the canonical Management Contract state before its Activation Block. The record includes the algorithm metadata, bytecode reference, gas parameters, ABI descriptors, and Activation Block. Activation therefore selects a proposal already present in the committed history rather than registering or modifying one at the height at which it becomes active.

Four conditions define the trusted base. The underlying consensus protocol provides a common canonical chain to honest nodes. The committee correctly authorizes upgrades and reviews both the algorithm implementation and its gas schedule before admitting metadata. Keccak256 is collision resistant. Finally, the configured WASM Runtime and controlled host interface implement deterministic ABI handling, WASM execution, host-interface responses, and initial instance state while enforcing resource limits, import validation, and error semantics. The runtime does not expose uncontrolled external state or host capabilities that can affect execution.

The adversary may choose arbitrary contract inputs, observe upgrade events, exploit asynchronous node-local preparation, and submit malformed or resource-intensive upgrade candidates, but admission remains subject to committee review and node-side validation. The model excludes a compromised Geth binary, a broken consensus protocol, a captured committee, failure of the designated implementation review, and a cryptographic primitive whose mathematical assumption is false. These assumptions define the claim boundary; EvoCrypt does not establish them.

4.2 Upgrade Security and Core Properties

Lemma 1 (Module binding). *Suppose an honest node prepares bytecode $\hat{C}$ for version (a, v) and accepts it only after checking $\text{Keccak256}(\hat{C}) = h_{a,v}$. Under collision resistance, $\hat{C}$ is computationally bound to the bytecode identified by the authoritative hash record. The hash check does not bind the ABI descriptors, gas parameters, or Activation Block: these fields are obtained from the authoritative Management Contract state and are interpreted by the version-selection rule.*

Proof. Hash recomputation over the recovered byte sequence is compared with $h_{a,v}$. If an adversary could cause two distinct byte sequences to pass this check for the same record, it would produce a collision in Keccak256. Therefore, except with negligible probability under the collision-resistance assumption, the prepared bytecode is the bytecode named by the recorded hash. The remaining metadata fields are not inputs to this comparison; the canonical Management Contract state fixes them. $\square$

Theorem 1 (Activation consistency). *For any two honest nodes with the same canonical chain and execution height b , the nodes select the same active version $v^*(a, b)$. For a revision registered before its Activation Block, a node*

that has completed preparation switches to the selected revision at and after that height, whereas a node whose preparation is incomplete continues attempting the update and does not select an obsolete version. Hence, node-local preparation time affects readiness but cannot change the canonical version selected for the same block.

Proof. The canonical chain determines the same Management Contract state, and therefore the same feasible set of records satisfying $H_{a,v} \leq b$. The lexicographic argmax in (2) is deterministic, so both nodes select the same pair $(H_{a,v}, v)$ and the same $v^*(a, b)$. The pre-activation registration assumption ensures that this record is already available in the committed history when the height-based rule selects it. Preparation affects whether a node is ready to execute the selected version, but it does not affect version selection. A node that is not ready continues its preparation procedure rather than using an obsolete implementation. Thus local preparation differences change readiness timing, not the version selected by the canonical rule. $\square$

4.3 Invocation Security

Theorem 2 (Invocation consistency). *Consider two honest nodes processing the same invocation at block height b . If they have the same pre-state, request (a, x) , gas budget, canonical Management Contract state, prepared bytecode, ABI descriptors, host-interface responses, and initial module state, then they produce the same ABI result or protocol error and charge the same gas $G(a, v^*(a, b), x)$.*

Proof. Activation consistency gives both nodes the same $v^*(a, b)$. Equal authoritative metadata then gives them the same ABI descriptors, gas parameters, and value of (3). Common ABI rules yield the same decoded input and encoded output shape.

Given the same bytecode, deterministic WASM execution, deterministic host-interface responses, and identical initial instance state, the module execution is the same transition from the same module state and input. If execution succeeds, both nodes return the same result; if it violates the specified runtime or protocol conditions, both nodes return the same protocol error. $\square$

Corollary 1 (State-transition consistency). *Under the hypotheses of the invocation-consistency theorem, the two nodes apply the same EVM state transition for the invocation, including the same gas charge and success or failure outcome.*

Proof. The invocation protocol resolves the version and gas requirement before executing the module. The theorem establishes equality of the selected version, decoded request, gas charge, module result or protocol error, and encoded return value. The EVM therefore receives the same execution outcome from the coprocessor while starting from the same pre-state. Deterministic EVM execution then yields the same post-state. $\square$

Proposition 1 (Conditional execution isolation). *Let $\mathcal{R}_{\text{prot}}$ be the set of host resources outside the resources explicitly authorized for a call, including Geth process memory other than the designated input and output buffers, local files, network sockets, and mutable host state not exposed by the coprocessor interface. If the WASM Runtime enforces bounded linear memory and validates imports against an allowlist, and if each permitted host function exposes no capability to access $\mathcal{R}_{\text{prot}}$, then an admitted module has no modeled execution path to read or modify $\mathcal{R}_{\text{prot}}$.*

Proof. A module can affect its environment only through its own linear memory and the host functions made available to it. The memory boundary prevents direct pointer-based access to the Geth address space, while import validation removes undeclared host capabilities. By assumption, the remaining host functions do not provide indirect access to protected resources. The conclusion is therefore a conditional isolation claim about the runtime boundary. It does not imply that the approved algorithm is free of a cryptographic backdoor, that its mathematical security is established, or that resource exhaustion is impossible within the configured limits. $\square$

4.4 Online Governance and Security Boundaries

Upgrade agility. After preparation succeeds, each honest node processing a block at or after the revision’s activation height uses the revision selected by the canonical version-selection rule. The same online governance path can update both the algorithm implementation and its gas parameters. No Geth restart is required because the runtime loads and activates the prepared module at execution time rather than requiring the implementation to be rebuilt into the client binary.

The governance claim is conditional. The analysis does not establish committee honesty, governance liveness when preparation cannot complete, end-to-end denial-of-service resistance, semantic equivalence between an uploaded module and its intended cryptographic specification, or security against a compromised client binary. The proofs establish consistency and conditional isolation under the stated assumptions, not cryptographic correctness of an approved implementation.

Collectively, the results give EvoCrypt a bounded security argument. Authoritative on-chain metadata and Activation-Block selection prevent version divergence under a common chain view. Deterministic runtime conditions extend consistency from version selection to invocation outcomes and EVM state transitions. The WASM boundary provides conditional host-resource isolation, while the governance path supports online algorithm and gas-schedule revisions without a client restart.

5 Evaluation

This evaluation asks whether EvoCrypt makes upgradeable cryptographic algorithms practical for contract-facing use. We measure the cost of making an algo-

rithm callable and the steady-state invocation cost, comparing the coprocessor path with Solidity and Precompiled Contract baselines where appropriate.

5.1 Implementation

Experiments used a Geth-based private Clique network running modified Geth v1.17.5-unstable, built with Go 1.25.7. Solidity contracts were compiled with solc 0.8.26, and WASM modules were built with TinyGo 0.42.0. Host hardware specifications remain to be recorded for full reproducibility.

The *network-wide upgrade duration* is the control-client-observable Geth-side setup time from upgrade submission until all evaluated nodes finish upgrade handling and the new algorithm is callable. The interval includes RPC submission, receipt observation, event handling, and node-local WASM preparation on each node, but excludes network propagation beyond the measured RPC path, consensus delay, and block-production waiting. With `period=0`, the network does not enforce an inter-block interval, so the metric reflects client-side upgrade processing rather than Clique block spacing.

Table 1 summarizes the eight contract-facing algorithms, their source or compiled artifact sizes on the three benchmark paths, and their cryptographic roles. Go source (B) denotes the reference implementation source size used by the Precompiled Contract path; WASM bytecode (B) denotes the uploaded module payload compiled from the same reference logic; Solidity runtime (B) denotes the compiled runtime bytecode of the deployed benchmark contract. The Go column therefore reports source size because the Precompiled Contract implementation is statically integrated into the client and does not produce an isolated, per-algorithm deployable artifact. These columns characterize implementation and deployment footprint rather than a strict binary-size comparison across all three paths. For Dh2048Secret and PedersenCommit, the Solidity runtime values include the linked shared modular-arithmetic library (8,524 B); estimated gas in Section 5.3 reflects the fully linked contracts. Add is a non-cryptographic baseline; the remaining algorithms cover hashes, key derivation, key exchange, commitments, signature verification, and finite-field polynomial multiplication.

Upgrade efficiency is measured by comparing the coprocessor upgrade path with Solidity contract deployment using mean setup latency. Execution efficiency is measured across EvoCrypt, Solidity, and Precompiled Contract paths after setup. The Precompiled Contract path uses the corresponding client-side implementations summarized in Table 1. Estimated gas denotes the gas estimate for a contract-facing call, not receipt-level `gasUsed`.

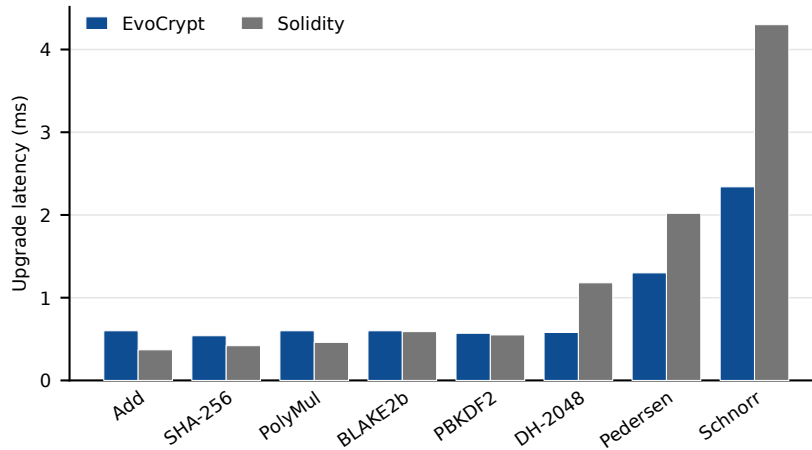
5.2 Upgrade Efficiency

Upgrade efficiency was measured by comparing the latency of making an algorithm callable through the coprocessor path with deploying an equivalent Solidity implementation. Figure 3 reports mean setup latency across all eight algorithms. The Precompiled Contract path is omitted because it requires no separate upload or deployment step in this setup.

Table 1. Source and compiled artifact sizes of the evaluated algorithms.

Algorithm	Go(B)	WASM(B)	Solidity(B)	Role
Add	156	11,287	172	Non-cryptographic baseline
Sha256	210	68,288	359	Cryptographic hash
PolynomialMul	790	7,987	1,005	Algebraic operation
Blake2bSum256	4,911	8,724	3,722	Cryptographic hash
Pbkdf2Sha256	1,377	72,580	1,556	Key derivation
Dh2048Secret	1,595	45,919	2,490 [†]	Key exchange
PedersenCommit	1,871	95,780	3,094 [†]	Commitment scheme
SchnorrVerify	2,171	95,960	4,233	Signature verification

[†]Solidity runtime bytecode includes the shared modular-arithmetic library linked at deployment.

**Fig. 3.** Mean upgrade latency (ms) across the evaluated algorithms.

The two paths were similar for lightweight algorithms. EvoCrypt required less setup time than Solidity for the heavier algorithms in the evaluated suite. For example, Add required 0.60 ms through EvoCrypt and 0.37 ms through Solidity, whereas SchnorrVerify required 2.34 ms and 4.30 ms, respectively. These measurements indicate that the coprocessor can make an algorithm callable without deploying a full Solidity implementation. Its setup path remained comparable to contract deployment for lightweight algorithms. The metric is the network-wide upgrade duration defined in Section 5.1. It covers Geth-side handling on all evaluated nodes and excludes network propagation, consensus delay, and unrelated-traffic availability.

5.3 Execution Efficiency

Steady-state invocation cost was evaluated after each algorithm became callable. Figure 4 compares mean invocation latency across EvoCrypt, Solidity, and Precompiled Contract paths; Figure 5 compares estimated gas for EvoCrypt and the Precompiled Contract path. Measurements used warm-up calls followed by repeated `eth_call` requests on the private-chain prototype.

The Precompiled Contract path was the fastest path across the evaluated algorithms. EvoCrypt remained close to Solidity on lightweight algorithms, but was faster than Solidity on the heavier cryptographic algorithms while remaining slower than the Precompiled Contract path. For the lightweight set, EvoCrypt latency was 0.50–0.62 ms. For SchnorrVerify, the corresponding means were 3.67 ms for EvoCrypt, 4.33 ms for Solidity, and 1.26 ms for the Precompiled Contract path. Thus, the relative performance of EvoCrypt depended on algorithm complexity rather than following a single ordering across all algorithms. The reported invocation latency values are means measured after warm-up on the evaluated private-chain prototype.

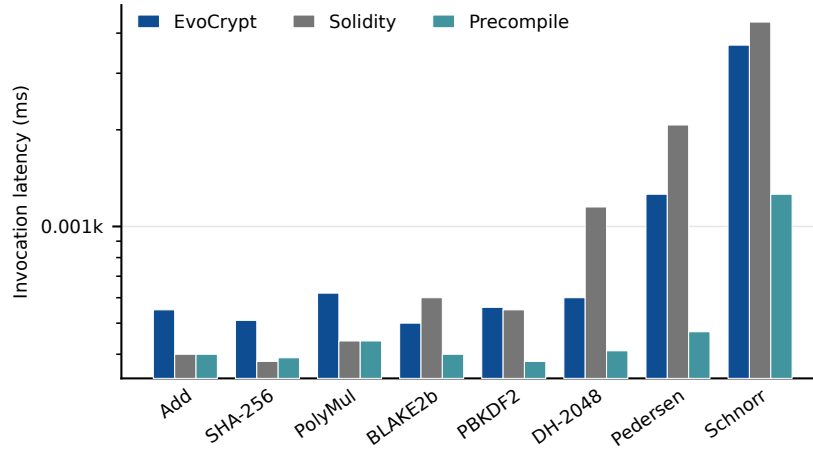


Fig. 4. Mean invocation latency (ms) for EvoCrypt, Solidity, and Precompiled Contract implementations.

Estimated gas was compared between the Precompiled Contract and EvoCrypt paths. SchnorrVerify illustrates the measured difference: the estimate was 2,301,691 gas for the Precompiled Contract path and 229,437 gas for EvoCrypt. For lightweight algorithms, the two paths remained within 13–15% of each other. These values represent experimental gas estimates for the two evaluated paths; they do not establish a calibrated economic schedule or a general magnitude of gas savings. The reported quantity is an estimate, not receipt-level `gasUsed`, and the registered prices are experimental parameters.

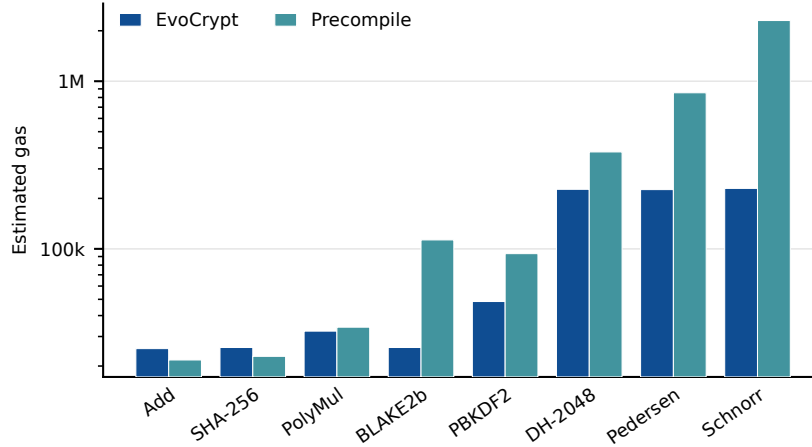


Fig. 5. Estimated gas for EvoCrypt and Precompiled Contract invocations.

Table 2. Invocation latency and uploaded WASM size for computationally intensive cryptographic algorithms.

Algorithm	EvoCrypt (ms)	Precompiled (ms)	WASM (B)
Aigis-Sig2Verify	1.299	1.030	72,213
Dilithium3Verify	1.589	1.100	26,679
ML-DSA-65Verify	1.565	1.052	70,422
SLH-DSA-SHAKE-192fVerify	19.458	6.241	19,339
Groth16Verify/BLS12-381	156.138	6.227	139,279
Groth16Verify/BN254	25.532	3.172	580,975

Complex-algorithm testing addressed the tension between deployment flexibility and EVM cost. The selected algorithms include post-quantum signatures and pairing-based zero-knowledge proof systems that are substantially more demanding than the regular contract-facing benchmark suite. We evaluated two dimensions: whether the algorithms could be uploaded, prepared, and invoked through a stable interface without a client modification, and what execution cost resulted relative to corresponding Precompiled Contract implementations. Table 2 reports invocation time and uploaded WASM size; times are in milliseconds and module sizes are in bytes.

EvoCrypt executed all algorithms in the stress experiment, including algorithms whose complexity and implementation size place them beyond practical direct Solidity realization in the evaluated setting and that are not present in the evaluated Precompiled Contract catalog. Successful execution supports the deployment-flexibility claim: such algorithms can be made callable through a stable interface without modifying the client for each algorithm.

A clear cost boundary remains. EvoCrypt latency was $1.3\text{--}25.1\times$ the corresponding Precompiled Contract latency, with the largest gap occurring for Groth16Verify/BLS12-381. EvoCrypt therefore addresses a deployment trade-off rather than providing a universal performance advantage. When a suitable Precompiled Contract already exists, the Precompiled Contract path remains preferable for execution efficiency.

6 Conclusion

EvoCrypt addresses the problem of evolving contract-facing cryptographic capabilities without embedding complex algorithms in EVM bytecode or modifying the client for every update. Its design separates the stable contract-facing interface from replaceable client-side WASM implementations, while deterministic version selection and algorithm-level accounting preserve consistent invocation rules.

Three bounded conclusions follow from the evaluated prototype. EvoCrypt provides an upgrade path with setup latency comparable to Solidity deployment for the evaluated algorithms, reports lower invocation gas estimates than the evaluated Precompiled Contract path under the experimental schedule, and makes complex algorithms callable when direct EVM realization is impractical. Execution remains slower than suitable Precompiled Contracts, and the current gas parameters are not economically calibrated. Future work will refine resource pricing and evaluate broader deployment and security conditions.

References

1. Ajmani, S., Liskov, B., Shriru, L.: Modular software upgrades for distributed systems. In: Thomas, D. (ed.) ECOOP 2006 – Object-Oriented Programming. Lecture Notes in Computer Science, vol. 4067, pp. 452–476. Springer, Berlin, Heidelberg (2006). https://doi.org/10.1007/11785477_26
2. Baresi, L., Ghezzi, C., Ma, X., Panzica La Manna, V.: Efficient dynamic updates of distributed components through version consistency. *IEEE Transactions on Software Engineering* **43**(4), 340–358 (2017). <https://doi.org/10.1109/TSE.2016.2592915>
3. Buterin, V., Reitwiessner, C.: EIP-197: Precompiled contracts for optimal ate pairing check on the elliptic curve alt_bn128. *Ethereum Improvement Proposals*, no. 197. <https://eips.ethereum.org/EIPS/eip-197> (2017), final. Accessed 21 August 2026
4. Cardano Foundation: About hard forks. <https://docs.cardano.org/about-cardano/evolution/about-hard-forks>, accessed 15 September 2026
5. Ciampi, M., Karayannidis, N., Kiayias, A., Zindros, D.: Updatable blockchains. In: Chen, L., Li, N., Liang, K., Schneider, S.A. (eds.) *Computer Security – ESORICS 2020*. Lecture Notes in Computer Science, vol. 12309, pp. 590–609. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-59013-0_29
6. Ethereum.org: Ethereum virtual machine (EVM). <https://ethereum.org/developers/docs/evm/>, accessed 21 August 2026

7. EVM Codes: Precompiled contracts. <https://www.evm.codes/precompiled>, accessed 21 August 2026
8. Grover, L.K.: A fast quantum mechanical algorithm for database search. In: Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing, pp. 212–219. Association for Computing Machinery, New York, NY, USA (1996). <https://doi.org/10.1145/237814.237866>
9. Huang, H., Xiang, C., Zhong, L., Zhou, Y.: PYLIVE: On-the-fly code change for python-based online services. In: 2021 USENIX Annual Technical Conference (USENIX ATC 21), pp. 349–363. USENIX Association, Berkeley, CA, USA (2021), <https://www.usenix.org/conference/atc21/presentation/huang>
10. Input Output: Combinator makes easy work of hard forks. <https://iohk.io/en/blog/posts/2020/07/23/combinator-makes-easy-work-of-hard-forks/> (2020), accessed 15 September 2026
11. Kumar, G., Saha, R., Conti, M., Buchanan, W.J.: LEAGAN: A decentralized version-control framework for upgradeable smart contracts. *IEEE Transactions on Services Computing* **18**(3), 1529–1542 (2025). <https://doi.org/10.1109/TSC.2025.3562323>
12. Nakamoto, S.: Bitcoin: A peer-to-peer electronic cash system. <https://bitcoin.org/bitcoin.pdf> (2008), accessed 21 August 2026
13. National Institute of Standards and Technology: Post-quantum cryptography. <https://csrc.nist.gov/projects/post-quantum-cryptography> (2026), updated 5 August 2026. Accessed 21 August 2026
14. Pina, L., Andronidis, A., Hicks, M., Cadar, C.: MVEDSUA: Higher availability dynamic software updates via multi-version execution. In: Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, pp. 573–585. Association for Computing Machinery, New York, NY, USA (2019). <https://doi.org/10.1145/3293882.3330551>
15. Polkadot: Runtime upgrades. <https://docs.polkadot.com/parachains/maintain/runtime-upgrades/>, accessed 15 September 2026
16. Reitwiessner, C.: EIP-196: Precompiled contracts for addition and scalar multiplication on the elliptic curve alt_bn128. Ethereum Improvement Proposals, no. 196. <https://eips.ethereum.org/EIPS/eip-196> (2017), final. Accessed 21 August 2026
17. Rodler, M., Li, W., Karame, G.O., Davi, L.: EVMPatch: Timely and automated patching of ethereum smart contracts. In: 30th USENIX Security Symposium (USENIX Security 21), pp. 1289–1306. USENIX Association, Berkeley, CA, USA (2021), <https://www.usenix.org/conference/usenixsecurity21/presentation/rodler>
18. Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Review* **41**(2), 303–332 (1999). <https://doi.org/10.1137/S0036144598347011>
19. Wang, C., Li, P., Fan, X., Tang, Z., Zeng, Y., Sakurai, K.: Phoenix: A live upgradeable blockchain client. *IEEE Transactions on Sustainable Computing* **8**(4), 703–714 (2023). <https://doi.org/10.1109/TSUSC.2023.3282586>
20. Wood, G.: Ethereum: A secure decentralised generalised transaction ledger. <https://ethereum.github.io/yellowpaper/paper.pdf> (2014), ethereum Yellow Paper. Accessed 21 August 2026
21. World Wide Web Consortium: WebAssembly Core Specification. <https://www.w3.org/TR/wasm-core-2/>, w3C Candidate Recommendation Draft. Accessed 3 September 2026
22. Zhang, Y., Crowcroft, J., Li, D., Zhang, C., Li, H., Wang, Y., Yu, K., Xiong, Y., Chen, G.: KylinX: A dynamic library operating system for simplified and

- efficient cloud virtualization. In: 2018 USENIX Annual Technical Conference (USENIX ATC 18). pp. 173–186. USENIX Association, Berkeley, CA, USA (2018), <https://www.usenix.org/conference/atc18/presentation/zhang-yiming>
23. Zindros, D.: Soft power: Upgrading chain macroeconomic policy through soft forks. In: Bernhard, M., Bracciali, A., Gudgeon, L., Haines, T., Klages-Mundt, A., Matsuo, S., Perez, D., Sala, M., Werner, S. (eds.) *Financial Cryptography and Data Security. FC 2021 International Workshops. Lecture Notes in Computer Science*, vol. 12676, pp. 467–481. Springer, Berlin, Heidelberg (2021). https://doi.org/10.1007/978-3-662-63958-0_36
 24. Zou, W., Lo, D., Kochhar, P.S., Le, X.B.D., Xia, X., Feng, Y., Chen, Z., Xu, B.: Smart contract development: Challenges and opportunities. *IEEE Transactions on Software Engineering* **47**(10), 2084–2106 (2021). <https://doi.org/10.1109/TSE.2019.2942301>